

How to create a basic hardware design that can be used through the PYNQ Python interface

Written by Jaskin Kabir jkabir@charlotte.edu • 11/13/2025

The PYNQ Framework is a very useful tool that allows the ZYNQ Ultrascale+ MPSoC Processing System to interface with the programmable logic through simple Python code running in a Jupyter notebook. This 45-minute tutorial shows how to use Vivado to create a hardware configuration, and how to use the PYNQ interface to interact with that hardware.

Objective

By the end of this tutorial, you will have a basic 4-function ALU implemented in programmable logic, and Python code that can use that ALU to perform basic arithmetic. This is quite a bit of cost and effort just for a 4-function calculator, but the true goal here is to gain experience with the core functionality of the PYNQ Framework.

The ALU will interact with the PS as an AXI slave

- It will use 4 registers
 - Opcode
 - Operand 1
 - Operand 2
 - Result
- The opcode register will determine the operation to be performed on Operand 1 and Operand 2 with the following scheme

Opcode	Operation
0	Operand 1 + Operand 2
1	Operand 1 - Operand 2
2	Operand 1 * Operand 2
3	Operand 1 / Operand 2

Pre-Requisites

- Vivado installed with the AUP-ZU3 board files
- An AUP-ZU3 board with
 - The power adapter
 - A USB-C Cable

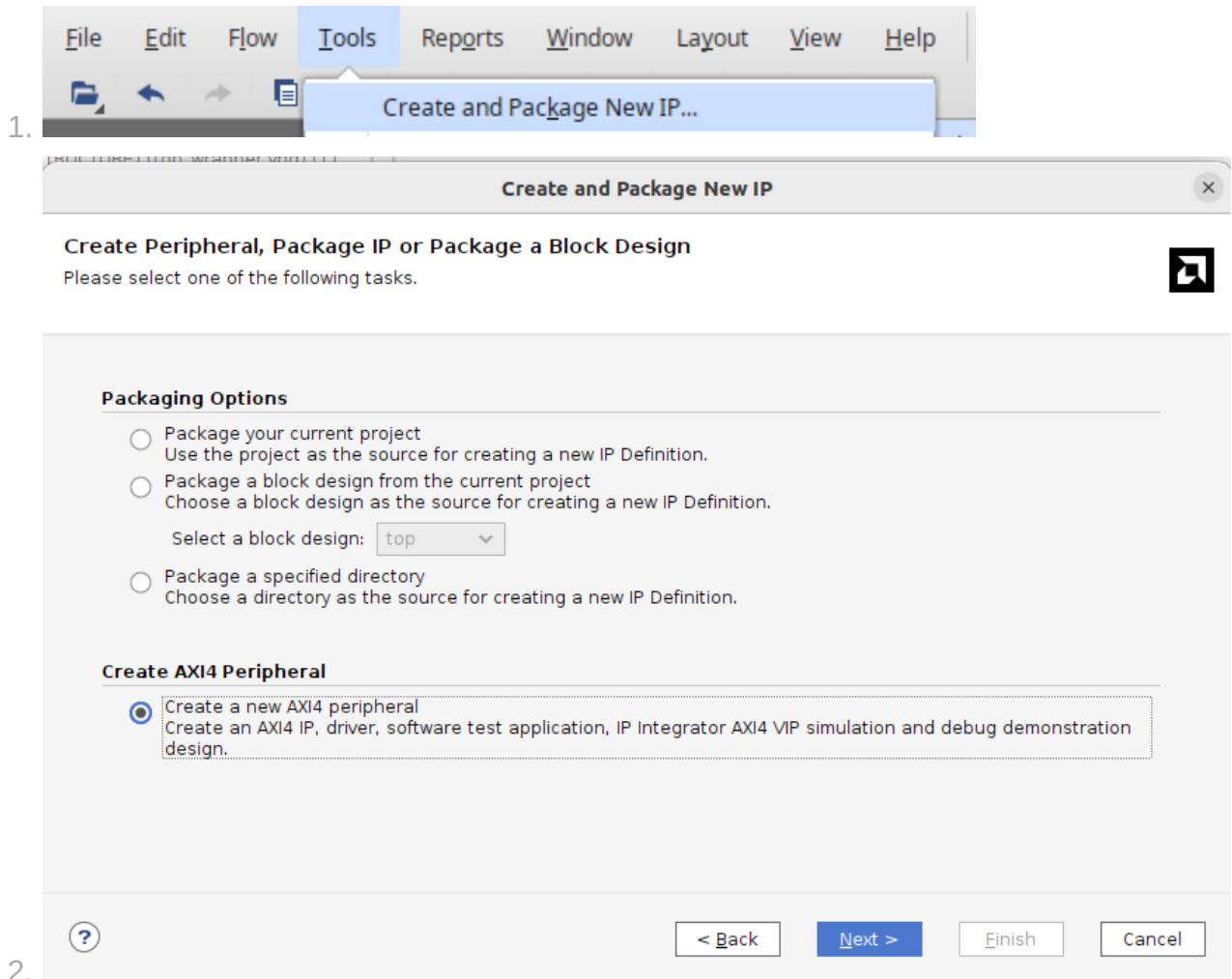
- A micro-SD card with the PYNQ Linux image

1. Creating The Hardware

1.1 Creating the IP

-

1. Create a new Vivado project targeting the AUP-ZU3
 1. This guide is written with VHDL, so to follow along make sure to set the target language of your project to VHDL
2. Create an IP with Vivado's create and package new IP wizard



Create and Package New IP

Add Interfaces

Add AXI4 interfaces supported by your peripheral

+

−

Interfaces

S00_AXI

...

S00_AXI

alu_v1.0

Name

S00_AXI

Interface Type

Lite

Interface Mode

Slave

Data Width (Bits)

32

Memory Size (Bytes)

64

Number of Registers

4

[4..512]

?

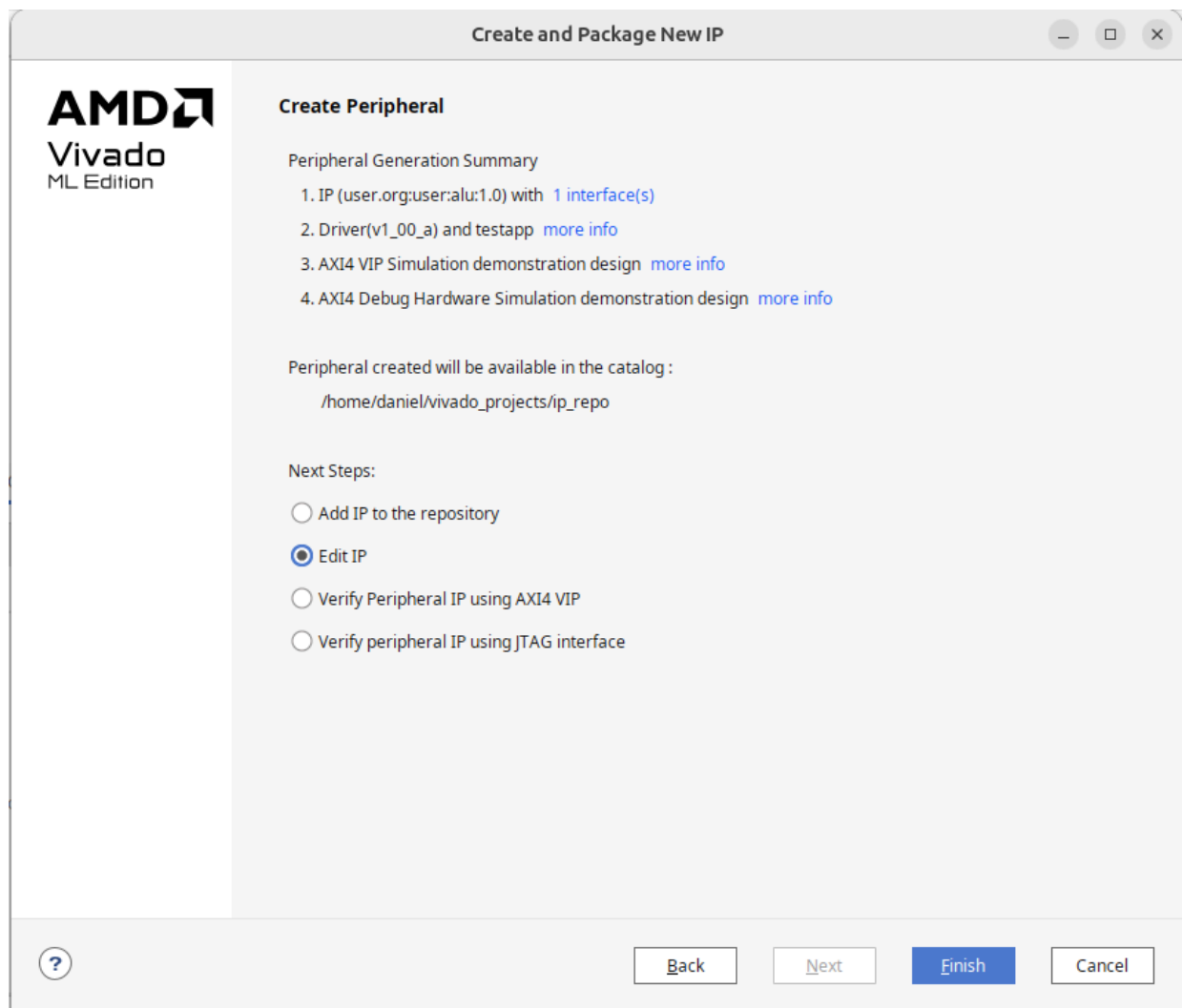
Back

Next

Finish

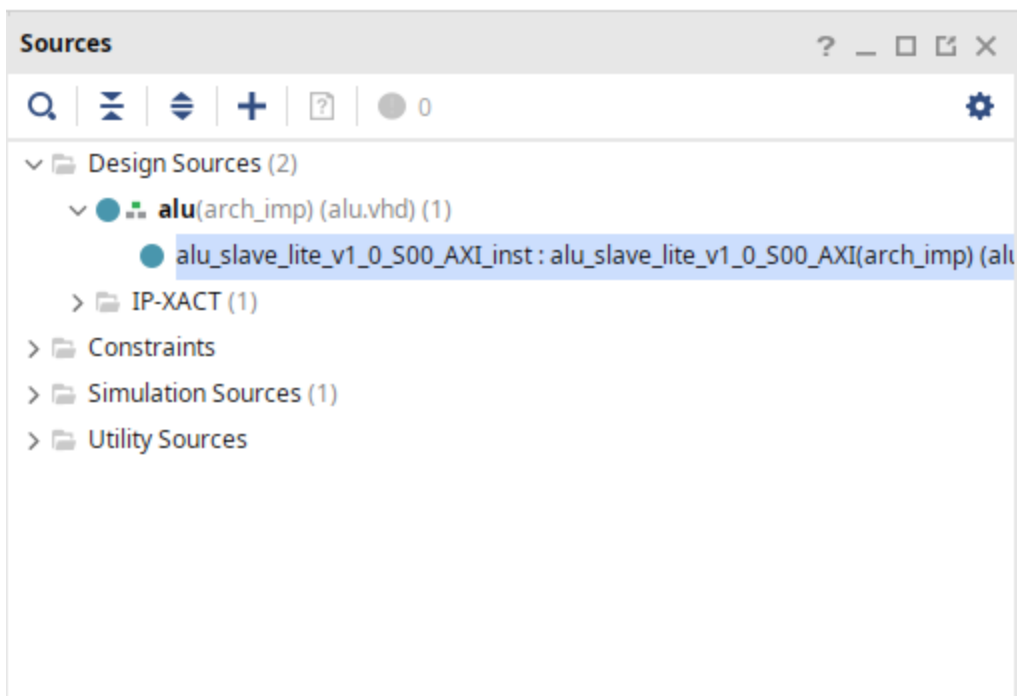
Cancel

3.



4.

3. Open the auto-generated HDL file shown below



1.

4. Comment out any lines that modify slave register 3, the result register, to ensure that it is read only

1. Line 216: `slv_reg3 <= (others => '0');`
2. Line 249: `slv_reg3(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7 downto byte_index*8);`
3. Line 256: `slv_reg3 <= slv_reg3;`
5. Create a signal to hold the result of the operation before it is moved into the result register
 1. Where the code says Signals for user logic register space example add this line:
 1. `signal alu_temp_result :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);`

```

---- Signals for user logic register space example
-----
---- Number of Slave Registers 4
signal slv_reg0 :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
signal slv_reg1 :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
signal slv_reg2 :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
signal slv_reg3 :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
signal alu_temp_result :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto
0);

```

6. Add this code to the end of the file where the code says Add user logic here

```

process (slv_reg0, slv_reg1, slv_reg2) begin
    case (slv_reg0(1 downto 0)) is
        when "00" =>
            alu_temp_result <= std_logic_vector(unsigned(slv_reg1) +
unsigned(slv_reg2));
        when "01" =>
            alu_temp_result <= std_logic_vector(unsigned(slv_reg1) -
unsigned(slv_reg2));
        when "10" =>
            alu_temp_result <=
std_logic_vector(resize(unsigned(slv_reg1) * unsigned(slv_reg2),
C_S_AXI_DATA_WIDTH));
        when "11" =>
            alu_temp_result <= std_logic_vector(unsigned(slv_reg1) /
unsigned(slv_reg2));
        when others =>
            alu_temp_result <= X"DEADBEEF";
    end case;
end process;

process (S_AXI_ACLK) begin

```

```

    if rising_edge(S_AXI_ACLK) then
        slv_reg3 <= alu_temp_result;
    end if;
end process;

```

1.1.1 VHDL Explanation

- Since you hijacked the write logic for register 3, you now have to write the code that implements the ALU operations and writes the result to register 3
- The code is split into two sections
- The combinational block:
 - The first process block reacts whenever registers 0-2 are changed, which are opcode, operand 1, and operand 2
 - It will then set the temporary result signal to the result of the operation specified by the opcode register
- The sequential block
 - The second process block is sensitive to the AXI clock
 - It will update the result register only on the rising edge of the clock signal
 - It is generally recommended to only write to AXI slave registers on the rising edge of the AXI clock, which is why the code must be split up this way

1.1.2 The Addressing and Memory Page

1. Before packaging the IP for integration in the block design, add descriptions of the 4 registers to the Addressing and Memory page of the IP packaging wizard.
 - This is important because this will be included in the hardware handoff (.hwh) file that will then be read by the Python API
2. Right click on the AXI interface and click **Add Register** once for each register in the design

1.

Name	Display Name	Description	Base Address	Range	Range Dependency
▼ S00_AXI			0	4096	

Name	Display Name	Value	Value Source
OFFSE		C_S00_AXI_BASEADDR	default
OFFSE		C_S00_AXI_HIGHADDR	default

3. Add names, descriptions, and crucially **Address Offsets** to the registers

1. Each register is 32 bits, or 4 bytes, so the addresses should be 4 addresses apart!

1.

Package IP - axi_alu

Packaging Steps

- Identification
- Compatibility
- File Groups
- Customization Parameters
- Ports and Interfaces
- Addressing and Memory**
- Customization GUI
- Review and Package

Addressing and Memory

Name: S00_AXI

Address Blocks

Name	Display Name	Description	Base Address	Range	Range Dependency	Width	Usage
S00_AXI_reg			0	4096		32	register

Address Block Parameters

Name	Description	Display Name	Value	Value Source	Parameter Types
OFFSET_BASE_P			C_S00_AXI_BASEAD	default	
OFFSET_HIGH_P			C_S00_AXI_HIGHAD	default	

Registers

Name	Display Name	Description	Address Offset	Size
opcode	opcode	0:add, 1:sub, 2:mul, 3:div	0	32
operand_1	operand_1	First operand	4	32
operand_2	operand_2	Second operand	8	32
result	result	Result of operation	12	32

4. Package the IP

1.

Project Summary x **Package IP - alu** x **alu_slave_lite_v1_0_S00_AXI.vhd** x

Packaging Steps

- Identification
- Compatibility
- File Groups
- Customization Parameters
- Ports and Interfaces
- Addressing and Memory
- Customization GUI
- Review and Package**

Review and Package

IP has been modified. 4 warnings 1 info message

Summary

Display name: alu_v1.0

Description: Basic 4 func alu

Root directory: /home/daniel/vivado_projects/ip_repo/alu_1_0

Edit IP Project Changes

Merge project changes to File Groups Wizard [Merge changes](#)

After Packaging

An archive will not be generated. Use the settings link below to change your preference

Project will be removed after completion

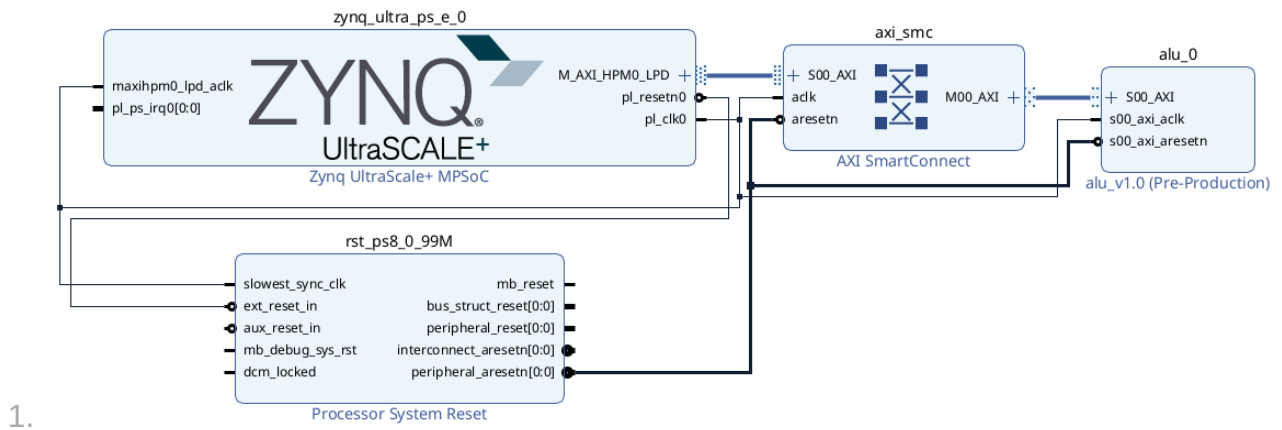
[Edit packaging settings](#)

[Re-Package IP](#)

1.2 Creating The Block Design

1. When you close the project for the ALU IP, it should drop you back into the project you created at the beginning
2. Create a new block design

3. Add a ZYNQ Ultrascale+ MPSoC IP Block, and the ALU IP
4. Run the block automation and the connection automation with default settings
5. After regenerating the layout, the final block design should look like this



1.3 Generating The Overlay Files

1. Create the HDL wrapper, generate the bitstream, and export the hardware as an XSA file
 1. Ensure the XSA file includes the bitstream

1.3.1 XSA File Explanation

- Within the XSA file are the .hwh and .bit files. The .bit file contains the hardware design and the HardWare Handoff (.hwh) describes the hardware interface
- The PYNQ system will use the XSA file to reprogram the FPGA fabric and define the interface

2. Writing The Software

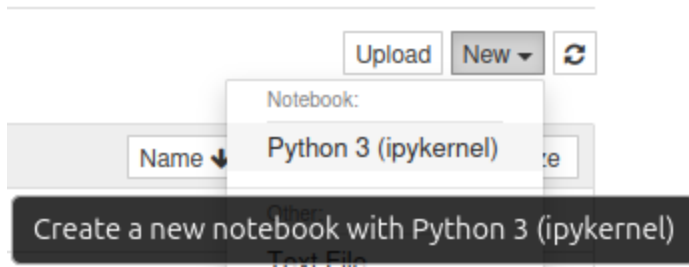
2.1 Transferring The Bitstream

1. Connect your PC to the ZU3 with a USB-C cable connected to one of the two ports labeled USB FLT
2. Open an SSH connection to the board with `ssh xilinx@192.168.3.1`
 1. The password is xilinx
 2. The IP may be different on your machine
3. Make a new folder for the ALU overlay with `mkdir /home/xilinx/pynq/overlays/alu`
4. Exit the ssh connection
5. Copy the xsa file with the command `scp [path_to_your_exported_xsa_file] xilinx@192.168.3.1:/home/xilinx/pynq/overlays/alu`

2.2 Using The Built-In Overlay Class

You can now instantiate PYNQ's built-in Overlay class using the XSA file you just generated and interface with the hardware through PYNQ's API

1. Open a browser and goto `192.168.3.1:9090`
 1. Or the IP of your board
2. The password is xilinx
3. Create a new notebook



1. Import the Overlay module and instantiate a new Overlay, pointing its constructor to the XSA you copied onto the device
5. Use the `help` function to see what IP cores are included in the overlay

```
from pynq import Overlay

overlay_path = "/home/xilinx/pynq/overlays/alu/alu.xsa"

overlay = Overlay(overlay_path)
help(overlay)
```

Help on Overlay in module pynq.overlay:

```
<pynq.overlay.Overlay object>
  Default documentation for overlay
/home/xilinx/pynq/overlays/alu/alu.xsa. The following
  attributes are available on this overlay:

  IP Blocks
  -----
  alu_0          : pynq.overlay.DefaultIP
  zynq_ultra_ps_e_0 : pynq.overlay.DefaultIP

  Hierarchies
  -----
  None

  Interrupts
```

```

-----
None

GPIO Outputs
-----
None

Memories
-----
PSDDR                : Memory
...

```

6. Python is able to recognize the ALU IP you added to the hardware design. Now, call the `help` function on its `reg_map` attribute to see how to interface with the IP core

```

alu_reg_map = overlay.alu_0.register_map
help(alu_reg_map)

```

Help on RegisterMapalu_0 in module pynq.registers object:

```

class RegisterMapalu_0(RegisterMap)
|   RegisterMapalu_0(buffer)
|
|   Method resolution order:
|       RegisterMapalu_0
|       RegisterMap
|       builtins.object
|
|   Data descriptors defined here:
|
|   opcode
|       Operation
|
|   operand_1
|       First Operand
|
|   operand_2
|       Second Operand
|
|   result
|       ALU Result
|
|   ...

```

7. As you can see, the PYNQ read the hardware handoff file and was able to read out the register descriptions you wrote earlier. Now, use the register map object to test the functionality of the ALU core

```
a = 30
b = 10

alu_reg_map.operand_1 = a
alu_reg_map.operand_2 = b

operations = [
    '+',
    '-',
    '*',
    '/'
]

for i in range(4):
    alu_reg_map.opcode = i
    print(f"{a} {operations[i]} {b} = {int(alu_reg_map.result)}")
```

```
30 + 10 = 40
30 - 10 = 20
30 * 10 = 300
30 / 10 = 3
```

2.3 Using the MMIO Module

- The PYNQ library also includes a module called `MMIO` (Memory Mapped I/O) to directly interface with registers by specifying their address space

1. First, find the ALU's base address with the `ip_dict` attribute of the Overlay instance

```
print(overlay.ip_dict)
```

```
{'alu_0': {'type': 'user.org:user:alu:1.0',
  'mem_id': 'S00_AXI',
  'memtype': 'REGISTER',
  'gpio': {},
  'interrupts': {},
  'parameters': {'C_S00_AXI_ADDR_WIDTH': '4',
```

```
'C_S00_AXI_DATA_WIDTH': '32',
'Component_Name': 'top_alu_0_0',
'EDK_IPTYPE': 'PERIPHERAL',
'C_S00_AXI_BASEADDR': '0x80000000',
...
```

2. You can now see that the ALU's base address is `0x80000000`. Now, create an MMIO instance with this base address, and a length of 16 (4 bytes per register * 4 registers)

```
from pynq import MMIO

base_addr = 0x80000000
alu_mmio = MMIO(base_addr, 16)

alu_mmio.write(0, 0) # Opcode 0 (+)
alu_mmio.write(4, 10) # Operand 1 = 10
alu_mmio.write(8, 30) # Operand 2 = 30

for i in range(0x0, 0xF, 0x4):
    print(f"Reading offset {hex(i)}")
    print(alu_mmio.read(i))
# Result (Offset 0xC) should be 10 + 30 = 40
```

```
Reading offset 0x0
0
Reading offset 0x4
10
Reading offset 0x8
30
Reading offset 0xc
40
```

2.4 Creating A Custom Overlay Class

- Rather than forcing the user to directly access the registers, a more user-friendly interface can be created by subclassing PYNQ's Overlay class.
1. Create a file called `alu.py` inside the `/home/xilinx/pynq/overlays/alu` directory with this code
 1. This class abstracts away the registers and exposes a single function for each operation

```

from pynq import Overlay

class AluOverlay(Overlay):
    def __init__(self, bitfile_name):
        super().__init__(bitfile_name)

        self.registers = self.alu_0.register_map

    def _operate(self, a, b):
        self.registers.operand_1 = a
        self.registers.operand_2 = b
        return int(self.registers.result)

    def add(self, a, b):
        self.registers.opcode = 0
        return self._operate(a,b)

    def sub(self, a, b):
        self.registers.opcode = 1
        return self._operate(a,b)

    def mul(self, a, b):
        self.registers.opcode = 2
        return self._operate(a,b)
    def div(self, a, b):
        self.registers.opcode = 3
        return self._operate(a,b)

```

2. Create a file in the `alu` directory called `__init__.py` that contains this single line: `from .alu import AluOverlay`.

1. This is to make the import logic more readable later

3. The final folder should look like this

```

xilinx@pynq:~/pynq/overlays/alu$ tree
.
├─ alu.py
├─ alu.xsa
└─ __init__.py

```

4. Now you can interface with the hardware through this abstracted class:

```

from pynq.overlays.alu import Alu0overlay

overlay_path = "/home/xilinx/pynq/overlays/alu/alu.xsa"
alu = Alu0overlay(overlay_path)
print(alu.add(2,2))
print(alu.sub(3,1))
print(alu.mul(5,6))
print(alu.div(15,5))

```

```

4
2
30
3

```

3. Included Files

- The code used in this guide is included in the Code directory of this archive
- The ip_repo directory contains the Vivado files for the ALU ip.
- The Python folder contains the pre-exported XSA file and the Python driver code to interface with it

```

Code
|   |--- ip_repo
|   |   |--- alu_1_0
|   |       |--- bd
|   |           |--- bd.tcl
|   |           |--- component.xml
|   |           |--- drivers
|   |               |--- alu_v1_0
|   |                   |--- data
|   |                       |--- alu.mdd
|   |                       |--- alu.tcl
|   |                   |--- src
|   |                       |--- alu.c
|   |                       |--- alu.h
|   |                       |--- alu_selftest.c
|   |                       |--- Makefile
|   |           |--- example_designs
|   |               |--- bfm_design
|   |                   |--- alu_tb.sv
|   |                   |--- design.tcl
|   |               |--- debug_hw_design
|   |                   |--- alu_hw_test.tcl

```

```
|
|
|      └─ design.tcl
|      └─ hdl
|          └─ alu_slave_lite_v1_0_S00_AXI.vhd
|              └─ alu.vhd
|      └─ xgui
|          └─ alu_v1_0.tcl
└─ Python
    └─ alu
        └─ alu.py
        └─ alu.xsa
            └─ __init__.py
    └─ alu.ipynb
```